

Divide and Conquer

344-111 ชุติวิชาการโปรแกรมและขั้นตอนวิธี

Divide and Conquer

- ▶ **Divide** a problem into a number of subproblems that are **smaller** instances of the same problem
- ▶ **Conquer** the subproblems by solving them **recursively**. If the subproblem sizes are small enough, however, just solve the subproblems in a **straightforward manner**
- ▶ **Combine** the solutions to the subproblems into the solution for original problem

ex. find max

9 2 8 3 5 6
↑ ↑ ↑ ↑ ↑ ↑

max = 9

D & C

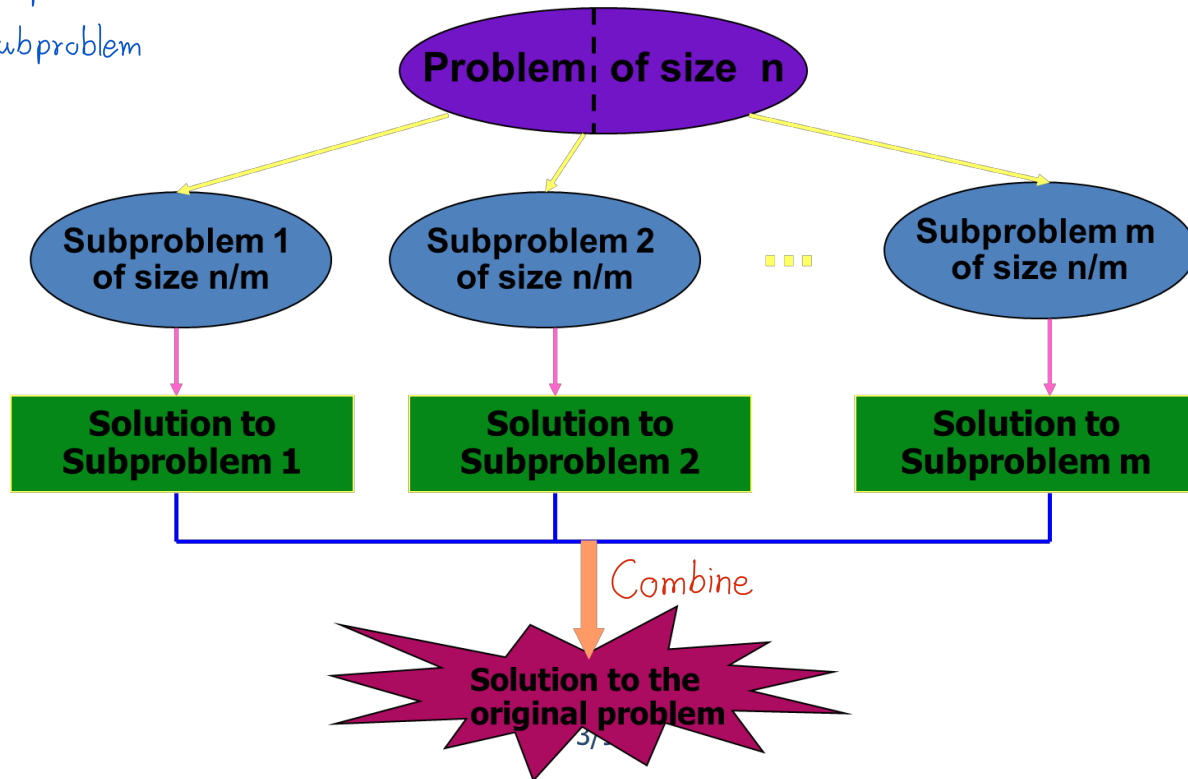
9 2 8 3 5 6

9 2 8 3 5 6

9 2 8 3 5 6
↓ ↓ ↓ ↓ ↓
9 8 5 6

Divide and Conquer

↓
Divide problem
to Subproblem



Divide and Conquer

Examples of problems solving by divide and conquer

- ▶ Merge Sort
 - ▶ Quick sort
 - ▶ Binary Search
- } In class
- ▶ Stassen's Matrix Multiplication
 - ▶ Large Integer Multiplication
- } Self-learning

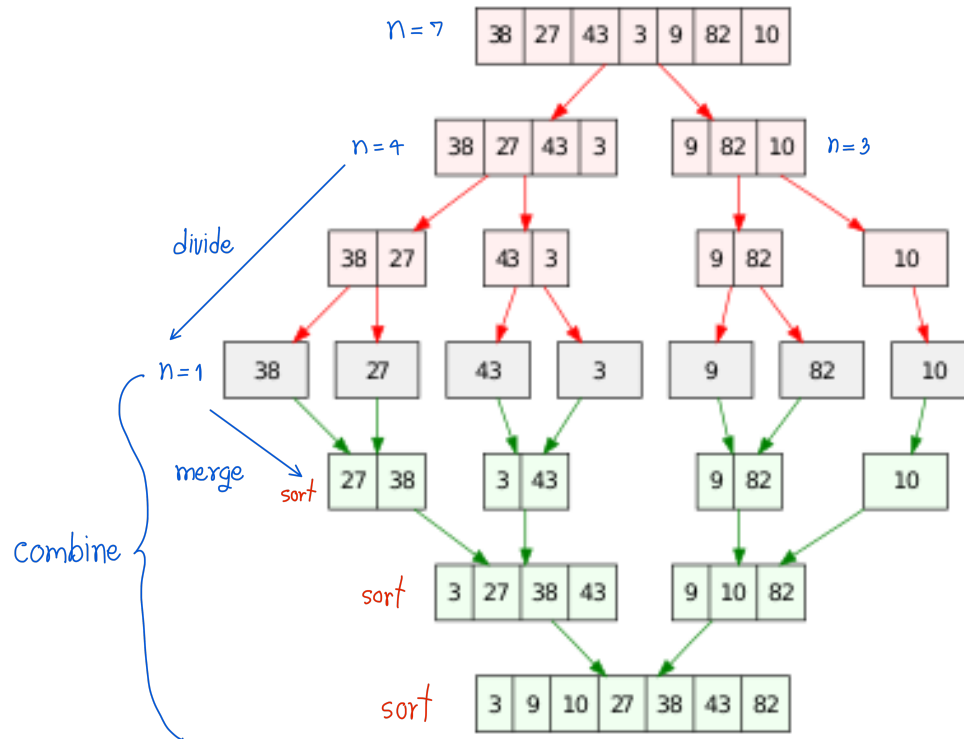


Merge Sort

- ▶ **Concept:**
- ▶ 1) **dividing the data into two subsets** of the same size
- ▶ 2) **sorting** each of them recursively
- ▶ 3) **merging the two smaller sorted** into the a single sorted.



Merge Sort



Algorithm: Mergesort

$A[0..n-1]$

$A[]$ (note)
left right

MergeSort (A, p, r)

{

if (p < r) q = (p+r)/2 $\sum^n A[0] \dots A[n-1]$ $A[p..q]$ $A[q+1..r]$

MergeSort (A, p, q)  } divide

MergeSort (A, q+1, r)

Merge (A, p, q, r) // merges $A[p..q]$ with $A[q+1..r]$ } merge

}

Time Complexity is $O(n \log n)$

Algorithm: Merge

MERGE(A, p, q, r)

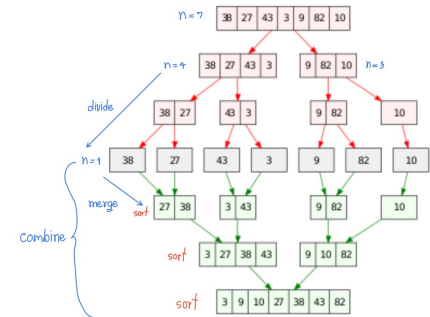
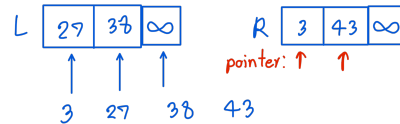
```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```

Prepare
copy memory ←

require sorted
↓
merge →



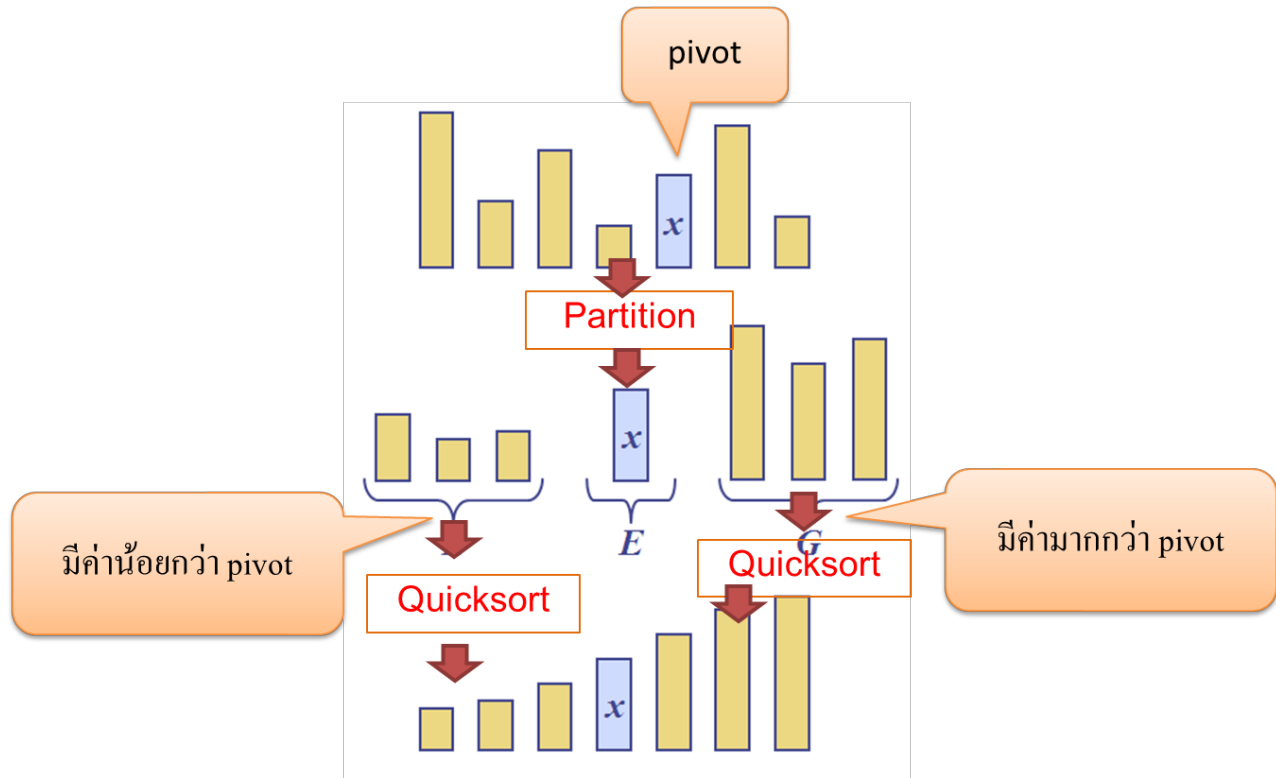
Time Complexity is $O(n)$

Quick sort

- ▶ **Concept:**
- ▶ 1) selecting a data, called **pivot**, from data points
- ▶ 2) **partitioning the data into two sets**; the left and the right of the pivot,
 - ▶ Left: the data that are smaller than the pivot
 - ▶ Right: the data that are greater than the pivot
- ▶ 3) recursive step 1) and step 2) on the two subsets independently



Quick sort



Algorithm: Quicksort

QuickSort(A, p, r)

```
{
  if (p < r){
    j = Partition(A, p, r)
    QuickSort(A, p, j)
    QuickSort(A, j+1, r)
  }
}
```

Partition(A, p, r)

```
{
  c = A[p]
  i = p-1; j = r+1
  while (i < j){
    while(A[--j] > c) {}
    while(A[++i] < c) {}
    if (i < j) swap(A[i], A[j])
  }
  return j
}
```

Time Complexity is $O(n \log n)$



Binary Search * ไม่สอบจ้จ

Concept:

- ▶ searching in a **sorted data**.
- ▶ Comparing a search key with the data in the middle position.
- ▶ If they match, the algorithm stop;
- ▶ Otherwise, the same operation is repeated recursively
 - ▶ for the left half of the array if the search key is smaller than the middle point,
 - ▶ for the right half of the array if the search key is greater than the middle point



Binary Search

Search = 37

1	2	3	4	5	6	7	8	9	10
12	15	18	23	26	37	39	41	43	48

mid

12	15	18	23	26	37	39	41	43	48
----	----	----	----	----	----	----	----	----	----

mid

12	15	18	23	26	37	39	41	43	48
----	----	----	----	----	----	----	----	----	----

mid



Algorithm: Binary Search

```
BinarySearch(A[left..right],x)
{
    if (left > right) return -1
    m = (left + right)/2
    if (x == A[m]) return m
    if (x < A[m])
        return BinarySearch(A[left..m-1],x)
    else
        return BinarySearch(A[m+1..right],x)
}
```

Time Complexity is $O(\log n)$



อ้างอิง

- ▶ Anany Levitin, Introduction to the Design and Analysis of Algorithm, second edition.
- ▶ Clifford A. Shaffer, A Practical Introduction to Data Structures and Algorithm Analysis, 2nd ED, Prentice Hall
- ▶ Thomas H. Cormen, Charles E. Leoserso, Ronald L. Rivest and Clifford Stein, Introduction to algorithm, Third Edition.